Q. What is fine-tuning in the context of LLMs, and how is it different from pretraining?

Fine-tuning is the process of adapting a pretrained LLM to a specific task, domain, or behavior by continuing training on a smaller, high-quality, task-aligned dataset.

In contrast, pretraining is where the model learns general language representations from massive, diverse corpora using self-supervised objectives like next-token prediction.

Q. When would you prefer fine-tuning over prompt engineering?

"I prefer fine-tuning when I need consistent, low-latency, and format-reliable outputs at scale, especially in cases like strict output format requirements (e.g., JSON/schema adherence) and high-throughput, cost-sensitive systems where long prompts become inefficient. Fine-tuning helps bake the behavior into the model, reducing prompt size, latency, and failure rates. However, if the problem can be effectively solved using prompt engineering or retrieval (RAG), I avoid fine-tuning due to its data dependency, risk of overfitting, and ongoing maintenance overhead."

Q. What are the risks of full fine-tuning in production systems?

- **Catastrophic Forgetting:** Model loses general capabilities and becomes overly specialized to fine-tuning data.
- **Overfitting:** Small or biased datasets cause poor generalization in real-world scenarios.
- **High Compute Cost:** Training and storing full model weights is significantly expensive compared to PEFT methods.
- **Slow Iteration & Rollback:** Each change requires retraining the entire model, making experimentation and recovery slow.
- **Deployment Complexity:** Managing multiple large model versions increases infra and maintenance overhead.

Q. How do you prevent overfitting during fine-tuning?

- **High-quality, diverse dataset:** Ensure data covers realistic variations and edge cases to improve generalization.
- **Validation monitoring + early stopping:** Track validation loss/metrics and stop training before the model starts memorizing.
- **Lower learning rate:** Use small LR (e.g., 1e-5 range) to avoid aggressive weight updates on limited data.
- **Regularization techniques:** Apply dropout and weight decay to reduce memorization.

- **Limit trainable parameters (PEFT):** Use LoRA/QLoRA instead of full fine-tuning to constrain model capacity.

Q. What is catastrophic forgetting, and how do you mitigate it?

Catastrophic forgetting is when a pretrained model, after fine-tuning on a narrow dataset, loses its general capabilities and becomes overly biased toward the fine-tuning distribution.

Ways to mitigate are –

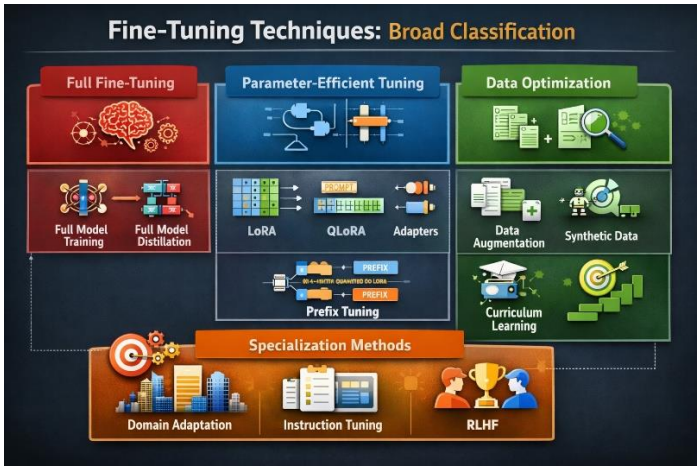
Use parameter-efficient fine-tuning (LoRA/QLoRA):
Freeze base weights and learn small adapters → preserves original knowledge.

Lower learning rate + controlled training:
Prevents large weight updates that overwrite pretrained representations.

Q. What are common failure modes after fine-tuning with their solution?

- **Catastrophic Forgetting → Use PEFT + data mixing:**
  Apply LoRA/QLoRA (freeze base) and include a small % of general/instruction data during training.
- **Overfitting → Improve data + early stopping:**
  Use diverse, high-quality data, monitor validation closely, and stop training early.
- **Format Overfitting → Increase prompt diversity:**
  Train on multiple prompt styles and variations instead of a single rigid template.
- **Instruction Degradation → Include instruction-tuning data:**
  Mix general instruction datasets so the model retains broad instruction-following ability.
- **Hallucinations → Add grounding signals (RAG / better data):**
  Improve dataset completeness or combine with retrieval to reduce unsupported generations.

Various Types of Fine Tuning Techniques are –


Fine-Tuning Techniques: Broad Classification
Full Fine-Tuning
Parameter-Efficient Tuning
Data Optimization
Full Model Training
Full Model Distillation
LoRA
QLoRA
Adapters
Data Augmentation
Synthetic Data
PREFIX
PREFIX
Prefix Tuning
Curriculum Learning
Specialization Methods
Domain Adaptation
Instruction Tuning
RLHF

LORA

LoRA (Low-Rank Adaptation) is a Parameter-Efficient Fine Tuning method that adapts large pretrained transformer models by freezing original weights and inserting small trainable low-rank matrices into important linear layers.

Training Flow of LoRA -

Step 1 — Load Pretrained Transformer

Start with a pretrained model having strong language understanding and general knowledge.

Step 2 — Freeze Base Parameters

All original weights are frozen (no updates) → preserves knowledge + reduces memory.

Step 3 — Inject LoRA Modules

Add low-rank adapters to selected linear layers → creates lightweight trainable paths.

Step 4 — Forward Pass

Output = Original output + LoRA correction → modifies behavior efficiently.

Step 5 — Loss Computation

Model predictions compared with ground truth using task-specific loss.

Step 6 — Backpropagation

Gradients flow only through LoRA parameters → base model remains unchanged.

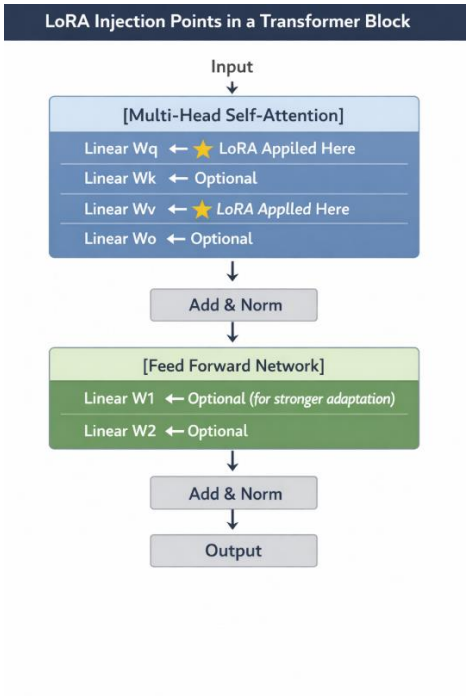
Step 7 — Parameter Update

Optimizer updates only LoRA matrices → fast and memory-efficient training.

Step 8 — Save Adapters

Store only small LoRA adapters → base model reused for multiple tasks.

Exact Locations Where LoRA Is Applied in Transformers –



Core Weight Update Rule (LoRA)

$$W' = W + \frac{\alpha}{r} BA$$

Practical Meaning

- W = frozen pretrained weight matrix
- A, B = small trainable low-rank matrices
- r = rank → controls adapter capacity
- α = scaling factor → controls update strength
- LoRA adds a learned correction (BA), not modifying W directly

Rank Selection Strategy

Rank (r) controls LoRA adapter capacity.

Low r = efficient but limited learning, High r = better learning but more memory.

◈ Practical Rank Guidelines

- r = 2–4 → small tasks (classification, low data)
- r = 8 → standard NLP tasks (default)
- r = 16 → complex generation tasks
- r = 32+ → heavy domain shift (medical/legal)

How Engineers Choose Rank

Start with low r → monitor validation → increase if underfitting.

Rule of Thumb

Small dataset + high rank → overfitting → match rank with task complexity.

LoRA Hyperparameters

- **Rank (r)** - Controls adapter capacity → low = underfitting, high = overfitting + memory waste
- **Alpha (α)** - Controls update strength → higher α = stronger adaptation (common: α ≈ 2r)
- **Target Modules** - Decide where LoRA is applied → typically Wq, Wv (FFN for stronger adaptation)
- **Dropout** - Regularizes adapters → prevents overfitting (typical: 0.05–0.1)
- **Learning Rate** - Training speed control → higher than full FT (typical: 1e-4 to 5e-4)

Q. What are failure cases of LoRA?

- **Limited Expressivity (low-rank bottleneck):**
  LoRA may fail on tasks requiring large behavioral shifts, as low-rank updates can't fully modify complex weight patterns.
- **Sensitivity to Hyperparameters:**
  Poor choice of rank, alpha, or target modules can significantly degrade results.
- **Layer Selection Issues:**
  Applying LoRA only to limited layers (e.g., attention only) may miss important transformations in MLP blocks.

Experienced teams treat LoRA as config-sensitive, not plug-and-play.

Q. How does LoRA affect inference latency?

LoRA has minimal to near-zero impact on inference latency because it only adds a small low-rank update to existing weight matrices, and in most production setups these updates are merged into the base weights at inference time.

In production, LoRA is widely preferred for latency-sensitive systems like chatbots and APIs because it introduces negligible inference overhead while maintaining efficiency. It enables adapter swapping, allowing multiple tasks or use cases to be handled without loading separate full models, which significantly reduces memory and deployment complexity.

QLORA

QLoRA (Quantized Low-Rank Adaptation) is an advanced Parameter-Efficient Fine-Tuning technique that combines 4-bit weight quantization with LoRA adapters to enable training extremely large language models using very limited GPU memory.

QLoRA Training Flow

- **Step 1** — Load Pretrained Model - Start with pretrained model in standard precision
- **Step 2 —** Quantize Weights - Convert weights to 4-bit → compress memory while preserving information
- **Step 3 —** Freeze Quantized Weights - Base model remains fixed → acts as knowledge backbone
- **Step 4** — Inject LoRA Adapters - Add low-rank matrices to selected layers → enable task learning
- **Step 5 —** Forward Pass - Quantized output + LoRA correction → modifies behavior efficiently
- **Step 6 —** Loss Computation - Predictions compared with labels using task-specific loss
- **Step 7 —** Backpropagation - Gradients update only LoRA → base weights unchanged
- **Step 8** — Save Adapters - Store quantized model + small LoRA adapters for deployment

Key Difference from LoRA

The base linear layers use quantized weights instead of full-precision weights. LoRA adapters remain full precision for stable learning.

Mathematical formulation –

QLoRA Weight Update Rule

$$W' = Q(W) + \frac{\alpha}{r} BA$$

Practical Meaning

- Q(W) - Frozen quantized pretrained weights (4-bit)
- Only A and B are trained → base quantized weights remain unchanged

Q. Why is 4-bit quantization used in QLoRA?

4-bit quantization in QLoRA is used to drastically reduce memory while maintaining training quality, enabling large models to be fine-tuned efficiently on limited hardware.

Why specifically 4-bit?

- **Memory Efficiency:**
  Reduces model size by ~4× compared to FP16 → enables single-GPU fine-tuning of large models (7B–65B)
- **Minimal Performance Drop:**
  Using NF4 (NormalFloat4) preserves distribution of weights better than naive INT4
- **Enables Backpropagation via Quantized Weights:**
  QLoRA uses double quantization + dequantization on-the-fly to keep training stable

Q. What is NF4 quantization, and why is it preferred?

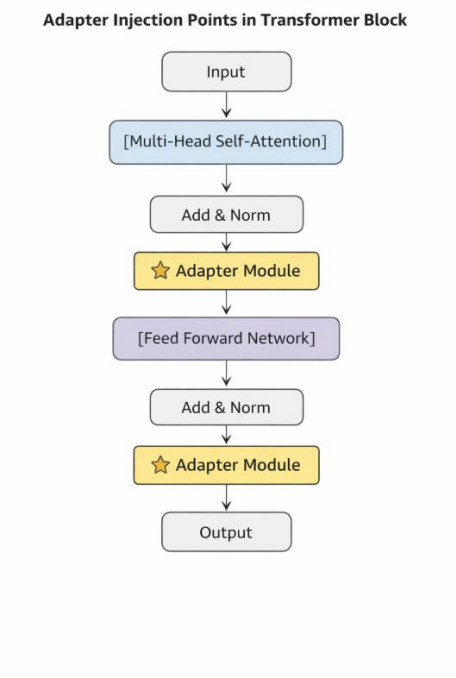
NF4 (NormalFloat4) is a 4-bit quantization scheme specifically designed for neural network weights, where values are mapped based on a normal (Gaussian) distribution rather than uniform spacing.

Why NF4 is preferred ?

- **Better accuracy than INT4:**
  Matches weight distribution → lower quantization error
- **Preserves model quality in QLoRA:**
  Enables stable fine-tuning even with aggressive 4-bit compression
- **Memory efficient:**
  Same 4-bit storage but higher effective precision

ADAPTERS

Adapters are Parameter-Efficient Fine-Tuning modules where small trainable neural networks are inserted inside transformer layers to adapt pretrained models for new tasks while keeping original weights frozen.



Adapter Training Flow

- **Step 1 — Load Pretrained Transformer** - Start with pretrained model having general language intelligence
- **Step 2 — Freeze Base Weights** - All parameters frozen → preserves knowledge + reduces memory
- **Step 3 — Insert Adapter Modules** - Add small bottleneck networks inside transformer layers
- **Step 4 — Forward Pass** - Hidden states pass through adapters → refine task-specific representations
- **Step 5 — Loss Computation** - Predictions compared with labels using task-specific objective
- **Step 6 — Backpropagation** - Gradients update only adapters → base model unchanged
- **Step 7 — Parameter Update** - Optimizer updates small adapter networks → fast and efficient
- **Step 8 — Save Adapter Weights** - Store only adapters → reused with base model for inference

Q. Can multiple adapters be combined at inference time?

Yes, multiple adapters (e.g., LoRA adapters) **can be combined at inference time**, but the effectiveness depends on how they are combined and whether they are compatible.

Ways to combine adapters -

- **Sequential switching (safe & common):**
  Load one adapter at a time per request → avoids conflicts
- **Weighted merging:**
  Combine multiple adapters as:
  W + ΔW□ + ΔW□ (optionally with scaling factors)
- **Adapter fusion (advanced):**
  Learn how to combine adapters via additional training

DORA

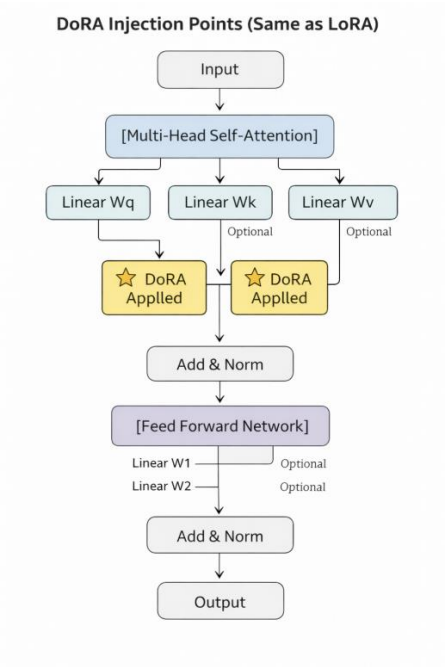
DoRA decomposes weights into magnitude + direction and fine-tunes only direction → better expressiveness and stability than LoRA.

◈ Core Idea

- Magnitude (norm) - Controls feature strength
- Direction - Controls information pattern
- DoRA updates direction only → preserves scale + improves learning capacity

Intuition

DoRA focuses on “where features point” (direction) instead of “how strong they are” (magnitude).



DoRA Training Flow

- **Step 1 — Load Pretrained Model** - Start with pretrained weights W
- **Step 2 — Decompose Weights** - Split into magnitude + direction components
- **Step 3 — Freeze Magnitude** - Keep norms fixed → preserve feature strength
- **Step 4 — Adapt Direction** - Apply low-rank updates to direction only
- **Step 5 — Forward Pass** - Recombine magnitude + updated direction
- **Step 6 — Loss Computation** - Compare predictions with labels
- **Step 7 — Backpropagation** - Update only direction adapters
- **Step 8 — Save Adapters** - Store DoRA adapters similar to LoRA

Mathematical Formulation

Weight Decomposition

$$W = m \cdot V$$

Meaning

- m - Magnitude (norm → feature strength)
- V - Direction (unit matrix → structural information)

Direction Update (DoRA)

$$V' = V + \frac{\alpha}{r} BA$$

Meaning

- A, B - Trainable low-rank matrices
- r - Rank → controls capacity
- α - Scaling factor → controls update strength
- Only direction (V) is updated → magnitude (m) remains fixed

Final Reconstructed Weight

After updating the direction the weight matrix is reconstructed = $m \cdot V'$

INSTRUCTION BASED FINE TUNING

Instruction-based fine-tuning is a training approach where an LLM is fine-tuned on (instruction/prompt → response) pairs so that it learns to follow natural language instructions and generate task-specific outputs in a consistent and controlled manner.

Core Idea

- Pretrained LLM = good text generator but poor task follower
- IFT teaches instruction → response mapping
- Transforms model into assistant-like system

Training Flow

- Step 1 — Load Pretrained LLM - Start with model having language understanding
- Step 2 — Build Dataset - Create instruction–response pairs
- Step 3 — Format Data - Structure as supervised examples
- Step 4 — Supervised Fine-Tuning - Train using teacher forcing
- Step 5 — Loss Computation - Cross-entropy between predicted and true tokens
- Step 6 — Backpropagation - Update model weights
- Step 7 — Behavior Emerges - Model learns assistant-style responses

Where Updates Happen

All major transformer parts updated → attention, FFN, layer norms, output layers (no architecture change).

Q. What is the role of prompt-response format in instruction tuning?

The prompt–response format is fundamental to instruction tuning, as it explicitly teaches the model how to map user instructions (prompts) to desired outputs (responses), aligning the model with real-world usage patterns.